
Rapport de Conception

NEXORA

Plateforme d'Intelligence Organisationnelle

Réalisé par :

Soubaibi Omar Yassine Baizi

Encadré par :

Mme. OUNACER SOUMAYA

Filière : 4IADM G5-A

Année Universitaire : 2025–2026

Table des matières

1	Introduction	2
2	Architecture Générale	2
2.1	Architecture 3-Tiers	2
2.2	Explication des Couches	2
3	Diagramme de Cas d'Utilisation	2
3.1	Description des Acteurs	2
3.2	Diagramme de Cas d'Utilisation	4
3.3	Cas d'Utilisation par Acteur	5
3.3.1	Utilisateur (Standard)	5
3.3.2	Manager (en plus des UC01–UC10)	5
3.3.3	Administrateur (en plus des UC01–UC14)	5
4	Diagramme de Classes	5
4.1	Diagramme de Classes	6
4.2	Relations entre Classes	6
5	Diagrammes de Séquence	7
5.1	Séquence : Authentification (Login)	7
5.2	Séquence : Recommandation d'Expert	8
5.3	Séquence : PageRank des Experts	10
6	Modèle de Données (Graphe Neo4j)	11
6.1	Schéma du Graphe	11
6.2	Propriétés des Nœuds	11
6.3	Pourquoi Neo4j ?	11
7	Conception des Algorithmes d'IA/ML	12
7.1	Pipeline ML Général	12
7.2	Algorithmes Implémentés	12
8	Conception du Déploiement	13
9	Conception de la Sécurité	14
9.1	Flux d'Authentification	14
9.2	Contrôle d'Accès (RBAC)	14

1. Introduction

Ce document présente la conception détaillée de la plateforme **Nexora**. Il décrit l'architecture logicielle, les diagrammes UML (cas d'utilisation, classes, séquences), le modèle de données en graphe, ainsi que les algorithmes d'IA/ML utilisés. L'objectif est de fournir une vue technique claire du fonctionnement interne du système.

2. Architecture Générale

2.1 Architecture 3-Tiers

L'application Nexora suit une architecture **3-tiers** classique, séparant la présentation, la logique métier et les données :

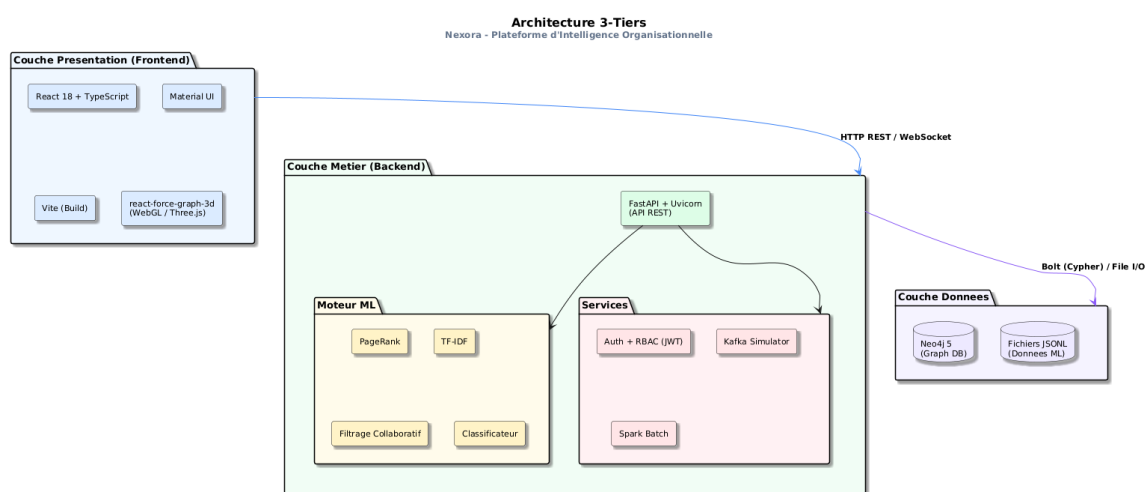


FIGURE 1 – Architecture 3-Tiers de Nexora

2.2 Explication des Couches

- **Couche Présentation** : Application SPA (Single Page Application) React servie par Nginx. Communique avec le backend via des requêtes REST (axios). Le graphe 3D utilise WebGL via Three.js pour le rendu temps réel.
- **Couche Métier** : API REST FastAPI qui contient toute la logique métier : authentification, RBAC, moteur de recommandation ML, pipelines Big Data. Le serveur ASGI Uvicorn gère les connexions HTTP et WebSocket (chat).
- **Couche Données** : Neo4j stocke les entités (Expert, Skill, Project, Document) et leurs relations dans un graphe. Les fichiers JSONL servent de source de données pour les modèles ML et les analyses Spark.

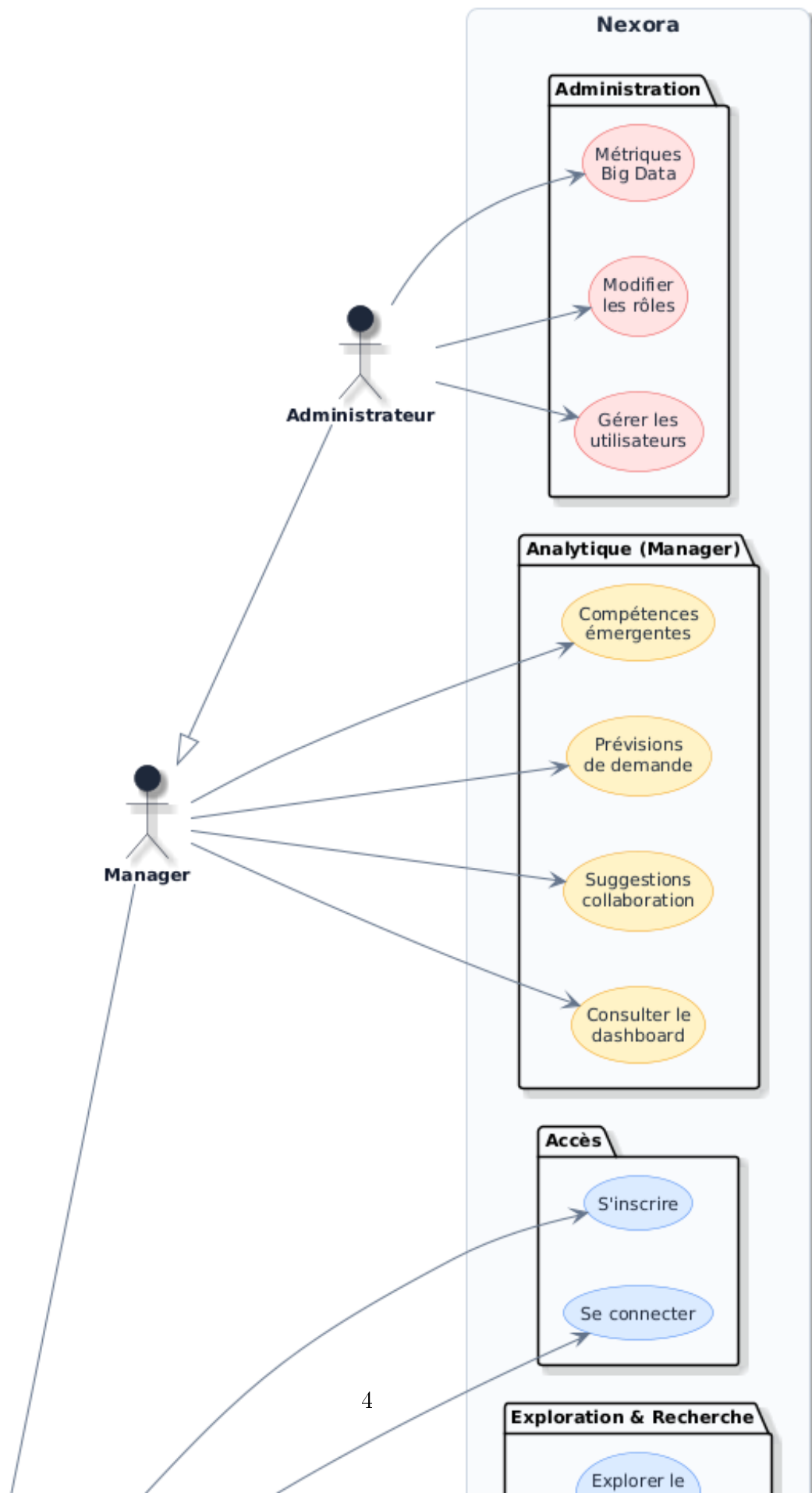
3. Diagramme de Cas d'Utilisation

3.1 Description des Acteurs

Acteur	Description
Utilisateur	Employé standard. Peut explorer le graphe, rechercher des experts, voir ses recommandations, communiquer.
Manager	Hérite de Utilisateur. Accède en plus aux prévisions de compétences, aux rapports analytiques et aux suggestions de collaboration.
Administrateur	Hérite de Manager. Peut en plus gérer les utilisateurs, modifier les rôles et accéder aux métriques Big Data (Kafka/Spark).

3.2 Diagramme de Cas d'Utilisation

Please use '!option handwritten true' to enable handwritten



3.3 Cas d'Utilisation par Acteur

3.3.1 Utilisateur (Standard)

ID	Cas d'utilisation	Description
UC01	S'inscrire	Créer un compte avec username, email et mot de passe.
UC02	Se connecter	S'authentifier pour obtenir un token JWT.
UC03	Explorer le graphe 3D	Naviguer dans la visualisation 3D du réseau organisationnel.
UC04	Rechercher un expert	Trouver un expert par compétence, département ou requête libre.
UC05	Voir recommandations	Consulter les experts recommandés par le moteur IA.
UC06	Poser une question à Veda	Interagir en langage naturel avec le chatbot IA.
UC07	Voir parcours d'apprentissage	Consulter les compétences suggérées et les lacunes identifiées.
UC08	Composer une équipe	Utiliser le Team Builder pour former une équipe optimale.
UC09	Communiquer (Chat)	Échanger dans les workspaces d'équipe via WebSocket.
UC10	Publier dans le fil	Partager des publications dans le feed.

3.3.2 Manager (en plus des UC01–UC10)

UC11	Voir compétences émergentes	Consulter les compétences en hausse et en déclin.
UC12	Voir prévisions de demande	Consulter les projections à 6–24 mois.
UC13	Voir suggestions collaboration	Voir les paires de départements complémentaires.
UC14	Consulter le dashboard	Voir les statistiques globales de l'organisation.

3.3.3 Administrateur (en plus des UC01–UC14)

UC15	Gérer les utilisateurs	Lister, rechercher et superviser les comptes.
UC16	Modifier les rôles	Changer le rôle d'un utilisateur (user/manager/admin).
UC17	Accéder métriques Big Data	Voir le statut du pipeline Kafka et les rapports Spark.

4. Diagramme de Classes

4.1 Diagramme de Classes

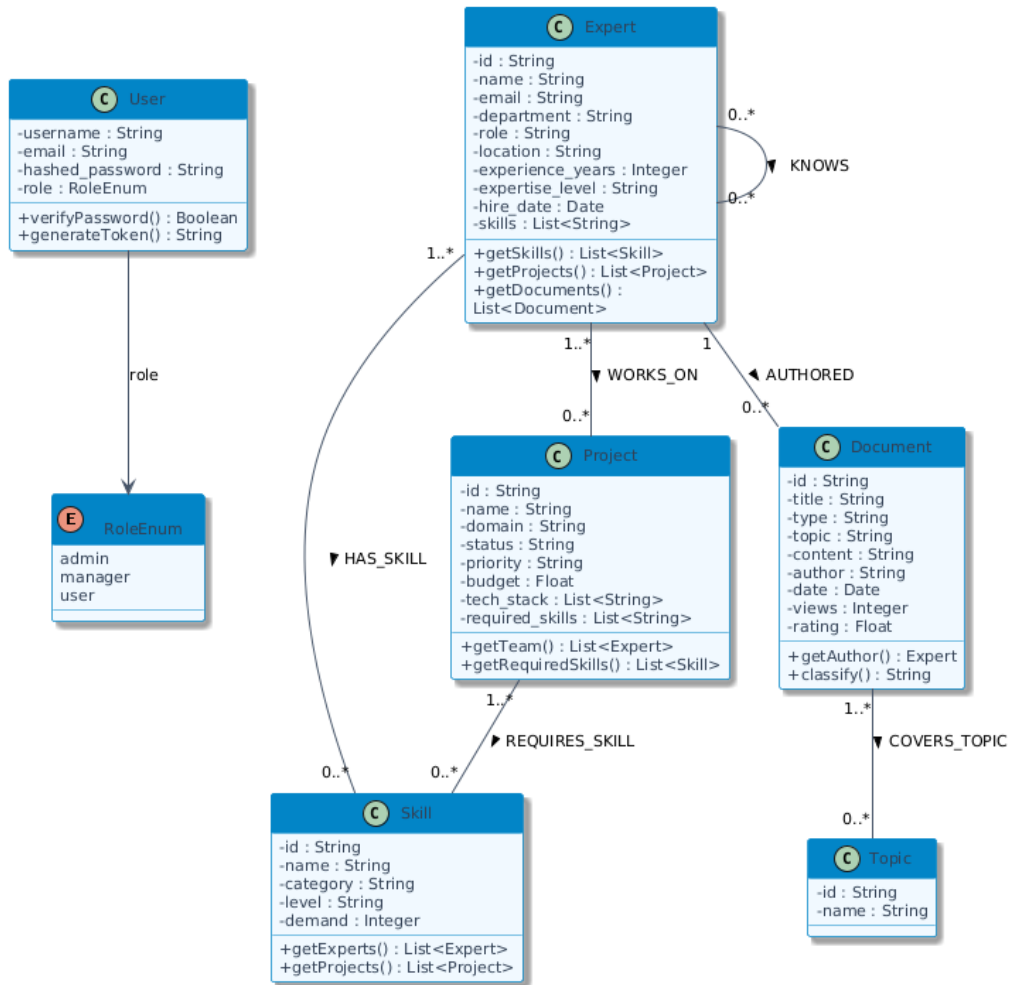


FIGURE 3 – Diagramme de classes — Nexora

4.2 Relations entre Classes

Les relations entre les classes sont les suivantes :

- Expert -HAS_SKILL-> Skill (N :M)
- Expert -WORKS_ON-> Project (N :M)
- Expert -AUTHORED-> Document (1 :N)
- Expert -KNOWS-> Expert (N :M)
- Project -REQUIRES_SKILL-> Skill (N :M)
- Document -COVERS_TOPIC-> Topic (N :M)

Explication : Les relations sont stockées nativement dans Neo4j sous forme d'arêtes orientées entre les nœuds, ce qui permet des parcours de graphe rapides (temps constant par relation, contrairement aux jointures SQL).

5. Diagrammes de Séquence

5.1 Séquence : Authentification (Login)

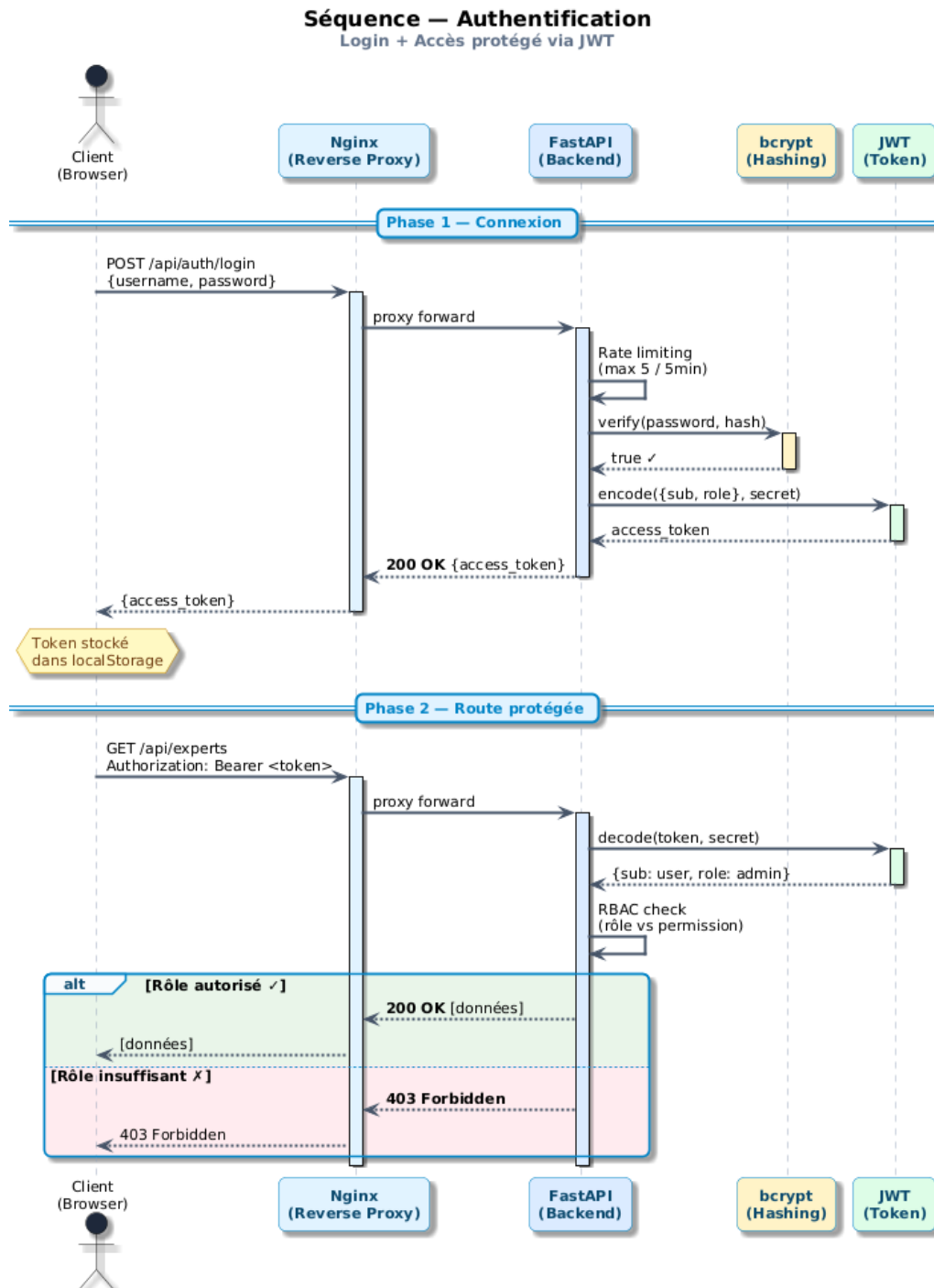


FIGURE 4 – Diagramme de séquence — Authentification

Explication :

1. Le client envoie les identifiants via POST.
2. Nginx redirige la requête vers FastAPI (proxy inverse).

3. FastAPI vérifie le mot de passe avec bcrypt (hachage + comparaison).
4. Si valide, FastAPI génère un JWT contenant le username et le rôle.
5. Pour les requêtes suivantes, le client envoie le token dans le header **Authorization**.
6. FastAPI décode le token et vérifie le rôle avant de répondre.

5.2 Séquence : Recommandation d'Expert

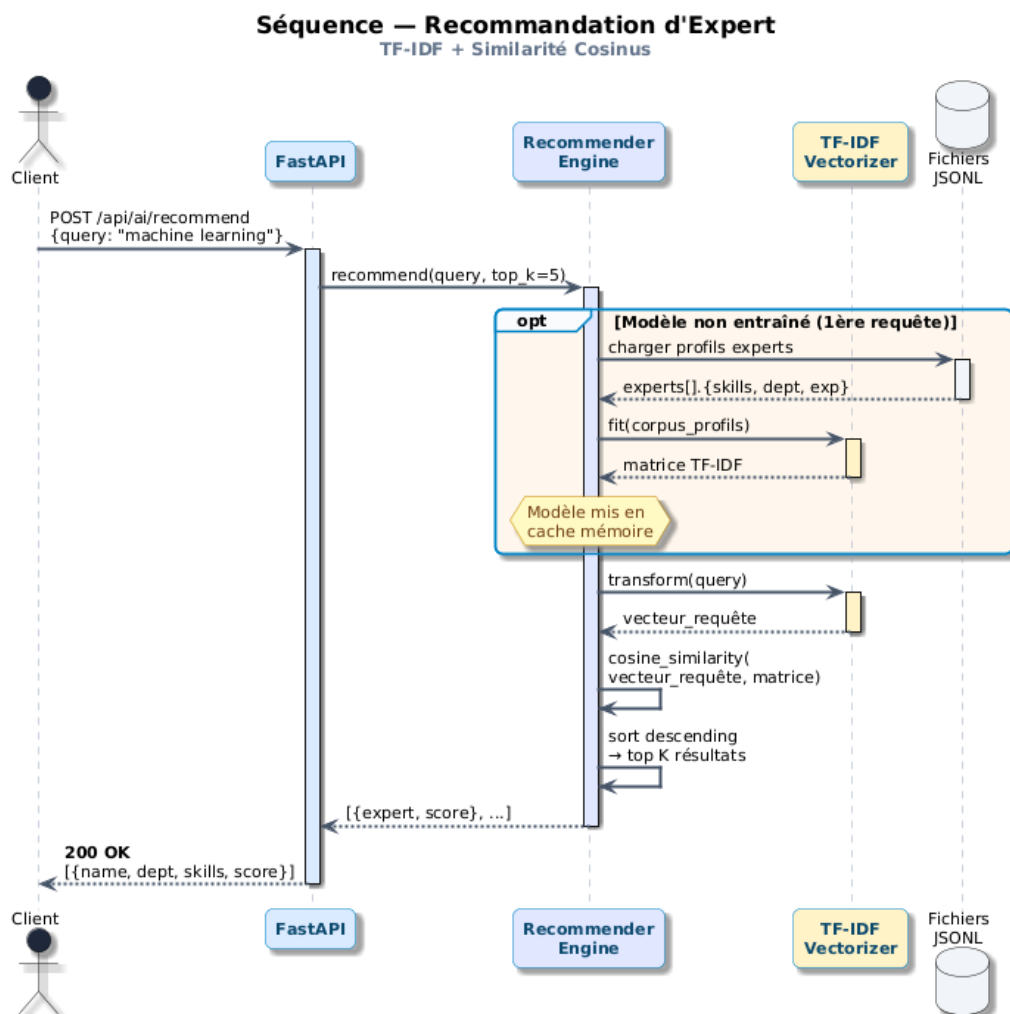


FIGURE 5 – Diagramme de séquence — Recommandation d'expert

Explication :

1. Le client envoie une requête textuelle décrivant le besoin.
2. Le Recommender charge les profils experts depuis les fichiers JSONL (si pas déjà en mémoire).
3. Les profils sont vectorisés via TF-IDF (Term Frequency–Inverse Document Frequency).
4. La requête est transformée dans le même espace vectoriel.
5. La similarité cosinus est calculée entre la requête et chaque profil.
6. Les K experts les plus similaires sont retournés avec leurs scores.

5.3 Séquence : PageRank des Experts

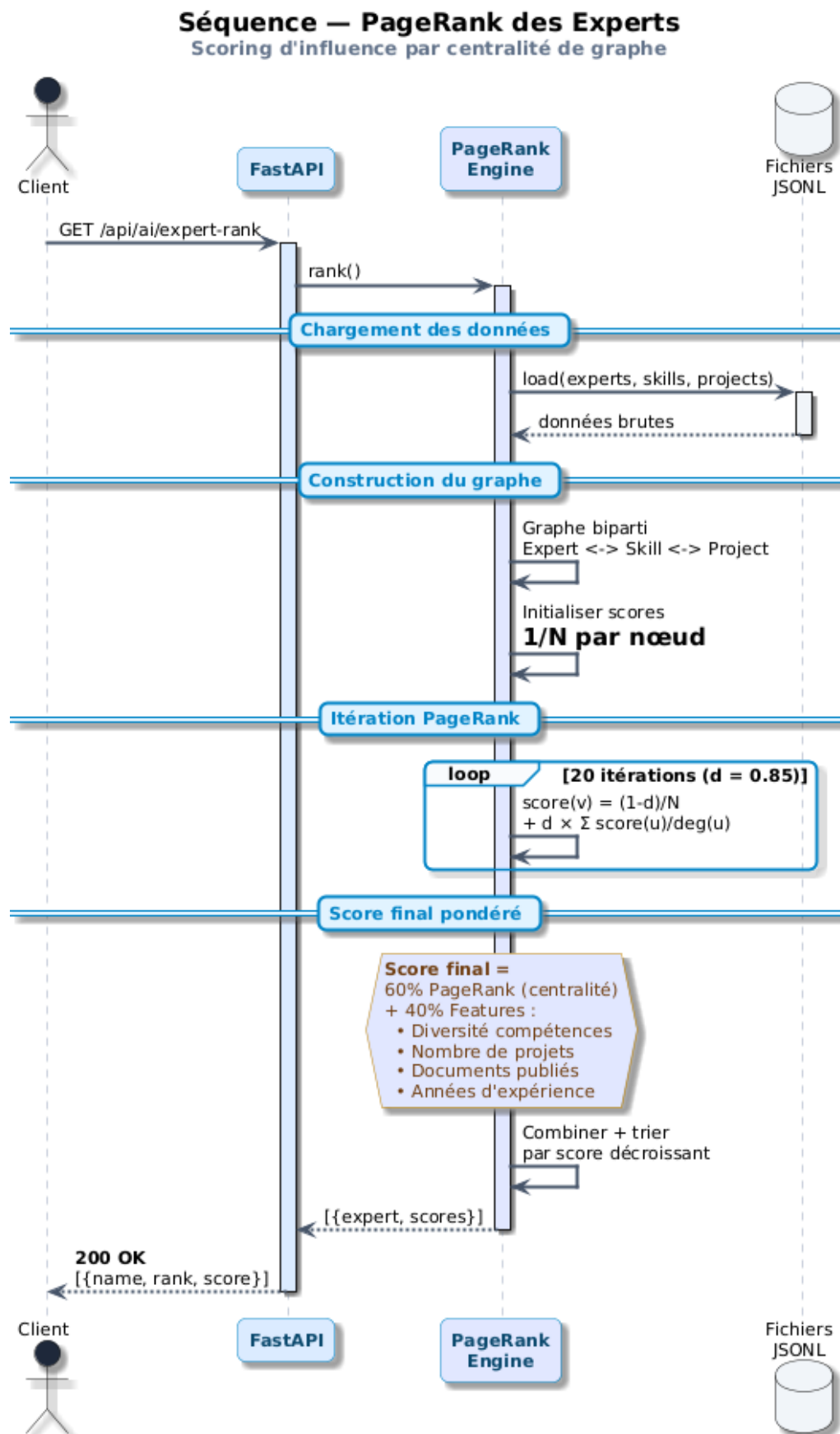


FIGURE 6 – Diagramme de séquence — PageRank des experts

Explication :

1. Un graphe biparti est construit à partir des données (Expert $\leftrightarrow$ Skill $\leftrightarrow$ Project).
2. L'algorithme PageRank itère 20 fois avec un facteur d'amortissement $d = 0.85$.
3. Le score final combine le rang PageRank (60%) avec des caractéristiques individuelles (40%) : diversité des compétences, nombre de projets, documents publiés, et années d'expérience.

6. Modèle de Données (Graphe Neo4j)

6.1 Schéma du Graphe

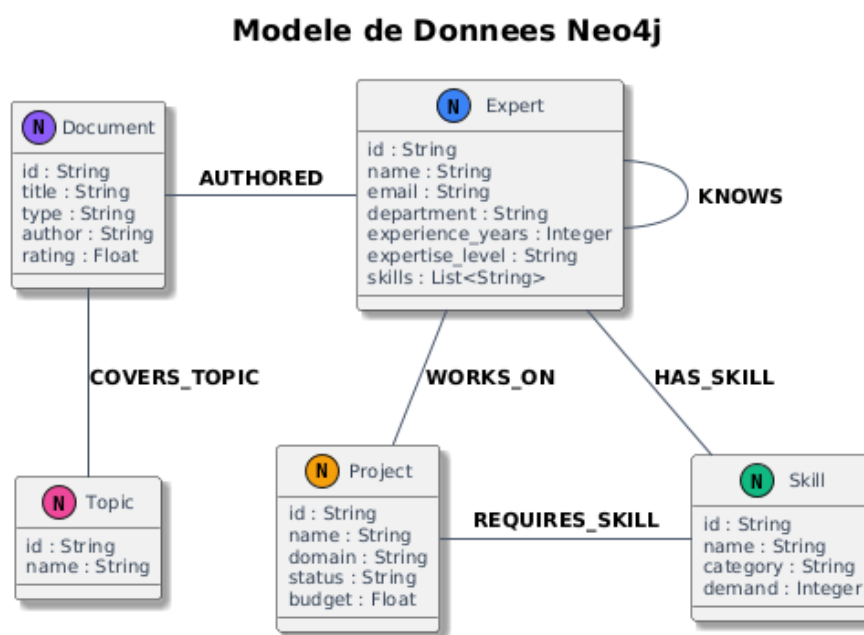


FIGURE 7 – Schéma du graphe — Modèle de données Neo4j

6.2 Propriétés des Nœuds

Nœud	Propriétés
Expert	id, name, email, department, role, location, experience_years, expertise_level, hire_date, skills[]
Skill	id, name, category, level, demand
Project	id, name, domain, status, priority, budget, tech_stack[], required_skills[], team_members[]
Document	id, title, type, topic, content, author, date, views, rating
Topic	id, name

6.3 Pourquoi Neo4j ?

- **Parcours de graphe efficaces** : trouver les experts liés à une compétence en temps constant (pas de jointures SQL coûteuses).

- **Langage Cypher** : requêtes déclaratives naturelles pour les patterns de graphe (MATCH (e:Expert)-[:HAS_SKILL]->(s:Skill)).
- **PageRank natif** : la structure de graphe est directement exploitable par les algorithmes de centralité.
- **Flexibilité du schéma** : pas de schéma rigide, facile d'ajouter de nouvelles relations.

7. Conception des Algorithmes d'IA/ML

7.1 Pipeline ML Général

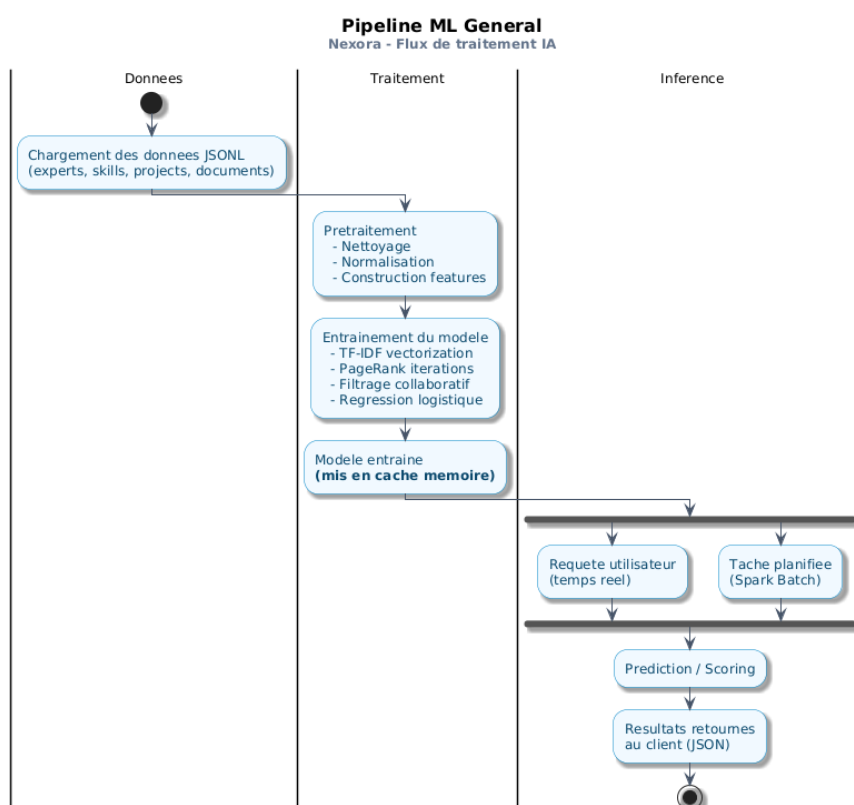


FIGURE 8 – Pipeline ML général — Nexora

7.2 Algorithmes Implémentés

Algorithme	Technique	Utilisation
Recommandation d'experts	TF-IDF + cosinus	Vectorise les profils et la requête, trie par similarité.
Scoring d'influence	PageRank itératif	Classe les experts par leur centralité dans le graphe.
Prédiction de lacunes	Filtrage collaboratif	Matrice user-skill, cosinus, recommande les skills manquants.

Compétences émergentes	Analyse de cohortes	Compare les taux d'adoption entre recrues récentes et anciennes.
Prévision de demande	Projection linéaire	Croise la demande projet $\times$ taux de croissance sur N mois.
Classification de docs	TF-IDF + Logistic Reg.	Classifie automatiquement les documents par catégorie.
Collaboration inter-dept.	Analyse ensembliste	Mesure la complémentarité des profils de skills entre départements.

8. Conception du Déploiement

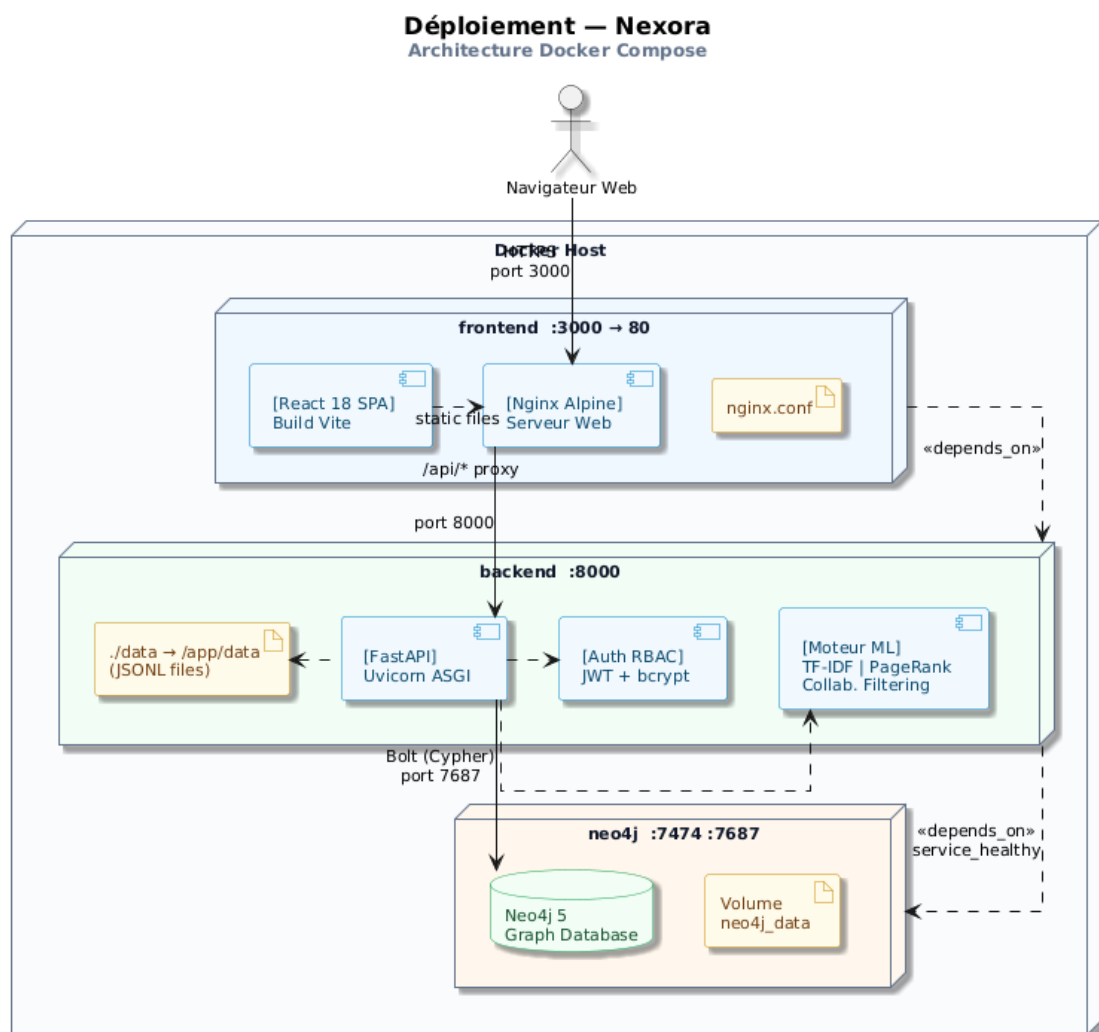


FIGURE 9 – Diagramme de déploiement — Docker Compose

Explication :

- **Neo4j** démarre en premier avec un healthcheck pour s'assurer qu'il est prêt avant le backend.

- **Le backend** ne démarre que quand Neo4j est *healthy*. Il monte le dossier `data/` pour accéder aux fichiers JSONL.
- **Le frontend** est compilé en production (Vite build), puis servi par Nginx qui gère aussi le proxy API vers le backend.

9. Conception de la Sécurité

9.1 Flux d'Authentification

1	L'utilisateur saisit son username et mot de passe sur la page de login.
2	Le frontend envoie POST <code>/api/auth/login</code> avec les identifiants.
3	Le backend vérifie le rate limiting (max 5 tentatives / 5 min).
4	Le mot de passe est comparé au hash bcrypt stocké.
5	Si valide : un token JWT est signé avec le username et le rôle.
6	Le token est stocké côté client (localStorage) et envoyé dans les headers suivants.

9.2 Contrôle d'Accès (RBAC)

Fonctionnalité	User	Manager	Admin
Explorer le graphe	✓	✓	✓
Rechercher des experts	✓	✓	✓
Chatbot Veda	✓	✓	✓
Prévisions compétences		✓	✓
Dashboard analytique		✓	✓
Gérer les utilisateurs			✓
Modifier les rôles			✓
Métriques Big Data			✓